

长上下文推理中的动态 KV Cache 压缩方案

KV260 复赛技术说明

芯溯猫猫队

2026 年 6 月 13 日

硬件平台：AMD Kria KV260

推理服务：llama-server / OpenAI-compatible API

模型配置：256M LoRA LLM + 原版视觉投影模块 mmproj

关键目标：降低 4096 token 长输出阶段 decode 的有效 KV 长度

目录

1	优化背景	2
2	参考论文方法	2
3	工程方案	2
4	压缩质量的量化方法	4
5	CUDA 大规模搜索结果	4
6	4096 Token 长输出压力测试	4
7	KV260 板端最终提交结果	5
8	为什么选择该策略	6
9	结论	7

1 优化背景

复赛任务中存在多类目标：短问答任务主要考察视觉理解与回答准确率；4096 token 长输出任务主要暴露长时间自回归生成时的 decode 吞吐瓶颈；能效和最终得分会受到 decode 时间、板端功耗和运行环境共同影响；TTFT 则主要反映图像编码、文本 prefill、服务端调度和首 token 前路径延迟。对于长输出，如果 LLM 在解码过程中一直保留完整上下文，attention 访问的 KV cache 会随生成长度线性增长，最终使 FPGA 端 decode 速度被长上下文拖慢。

因此，本部分优化只针对 LLM 解码阶段的 KV cache，不改变视觉编码器、mmproj/多模态投影和图像特征提取过程。目标是在不明显破坏短问答准确率和长输出可读性的前提下，把长输出阶段的有效 KV 窗口控制到约 128 token 量级，从而降低 decode 阶段 attention 访问长度和片上/片外访存压力。

说明范围：本节重点讨论该策略对 4096 token 长输出 decode 的影响；能效、TTFT 和综合分数在最终提交 JSON 中作为完整系统指标列出。

2 参考论文方法

我们参考了 Bhattacharya 等人在 IEEE ISCAS 2026 的会议论文 *A Graph-Based Methodology for Dynamic KV-Cache Compression in Transformer Inference* [1]。该论文的核心思想不是简单丢弃上下文，而是在运行过程中动态压缩 KV cache，将语义相近的 KV 条目合并成代表性的 centroid。

论文方法可以概括为：

- 保留开头少量 token 作为 attention sink。论文中固定 $N_{\text{sink}} = 4$ ，用于保持 attention 和 RoPE 位置编码的稳定性。
- 每隔固定生成间隔 M 对 KV cache 做一次压缩。
- 把可压缩区域中的 KV 条目看作图节点，用 key 向量的欧氏距离构图。
- 当两个 key 向量距离小于阈值 ϵ 时，在图中连边。
- 通过聚类/标签传播得到若干簇，每个簇的 K/V 取平均值作为 centroid。
- 压缩后的 KV cache 由 **sink tokens + centroids** 组成。

论文的评价指标包括压缩率、perplexity、吞吐率和延迟。其结论是：在 GPT-2 和 TinyLlama 上，该类 graph-based merge 可以达到很高的 KV cache 压缩率，同时保持相对稳定的 perplexity 和吞吐。论文也指出压缩频率 M 是一个重要折中：过于频繁会增加压缩开销，过于稀疏则无法及时降低 KV 长度。

该论文对本方案的启发主要有两点：

1. 前几个 sink token 不应被普通滑窗裁掉，它们对 RoPE 位置稳定性和全局 attention 分布有帮助。
2. 压缩策略不能只看 token/s，还必须量化压缩后输出质量是否退化。

3 工程方案

论文中的 graph merge 方法信息保留能力更强，但直接落到当前 llama.cpp + FPGA runtime 中成本较高。一方面，在线计算 key 向量距离和聚类会带来额外开销；另一方面，非连续 KV 布局 and 稀

疏 cache 在 llama-server 的 slot 复用路径中容易引入稳定性问题。因此，最终稳定提交线采用更直接、可验证的 server-side tail crop + physical compact:

```
compressed KV = first 4 sink tokens + recent tail tokens
```

该实现保留两个连续区域:

- **sink**: 保留最前面的 4 个 token，作为 RoPE 和 attention sink 锚点。
- **recent tail**: 保留最近 80 个 token，维持长输出过程中的局部语言连续性。
- **server bookkeeping**: 同步裁剪 server 侧 token 历史，并维护 media chunk 位置，避免多模态请求保留过期位置。
- **physical compact**: 把保留下来的 K/V tensor 行搬移到低地址 cache cell，使后端实际看到更短的 KV 使用范围。

当 LLM KV 长度超过阈值后，runtime 先删除中间区域，再把 tail 的位置整体前移，最后执行 seq_compact。这个物理 compact 很关键：只有实际降低 decode 阶段的 $\text{used_max_p1} / n_kv$ ，attention 才会真正访问短 cache，长输出 decode 才会直接改善。

模型配置为原版视觉投影模块 (mmproj) 和 256M LoRA LLM。这里的 256M 指 LLM/LoRA 侧规模，不指 mmproj。稳定提交线的目标运行参数如下:

```
export AICAS_SERVER_KV_TAIL_CROP=1
export AICAS_SERVER_KV_CROP_AFTER=128
export AICAS_SERVER_KV_CROP_KEEP=4
export AICAS_SERVER_KV_CROP_TAIL=80
export AICAS_SERVER_KV_PHYSICAL_COMPACT=1
```

其中:

- **AICAS_SERVER_KV_TAIL_CROP=1** 表示启用 server 侧 KV tail crop。
- **AICAS_SERVER_KV_CROP_AFTER=128** 表示 slot.n_past 超过 128 后触发裁剪。
- **AICAS_SERVER_KV_CROP_KEEP=4** 保留前 4 个 sink token。
- **AICAS_SERVER_KV_CROP_TAIL=80** 保留最近 80 个 token。
- **AICAS_SERVER_KV_PHYSICAL_COMPACT=1** 表示执行真实物理 compact。

裁剪后的有效长度约为 $4 + 80 = 84$ 。随后生成继续推进，KV 长度增长到超过 128 后再次裁剪，形成从约 84 到 129 再回到 84 的锯齿形有效窗口。对应源码包位于:

```
03_优化方法源代码/04_软件优化/02_长上下文KVCache压缩/
KVCache裁剪源码包/
```

其中 source/server/server.cpp 是裁剪入口，source/kv_cache/llama-kv-cache.cpp 是物理 compact 实现。

4 压缩质量的量化方法

论文使用 perplexity 衡量压缩后模型分布是否偏离原模型。但我们的长输出任务不是标准语言模型困惑度测试，而是要求模型生成满 4096 token。实际失败模式主要是提前停止、重复循环、乱码，或者输出被单一短语支配。因此，我们在论文指标基础上加入适合本任务的长输出质量指标。

```
Completion@4096 = generated_tokens / 4096

Repeat-n = max_count_of_any_n_gram / total_n_grams

Good output = completion_tokens == target
               and Repeat-4 <= 0.20
               and output is not dominated by one token/string pattern

Severe loop = Repeat-4 > 0.35
```

其中 Repeat-4 和 Repeat-8 用于衡量长文本是否进入重复循环。这个指标比人工观察更稳定，也能解释为什么某些配置虽然 token/s 很高，但实际输出不可用。

策略筛选时，短问答准确率作为约束指标单独检查：

```
Acc_opt >= Acc_ori - 5%
```

也就是说，短问答准确率用于证明视觉问答能力没有明显下降；长输出重复度和完成率用于证明上下文压缩没有把 4096 token 生成变成无意义重复。

5 CUDA 大规模搜索结果

为了减少在 KV260 上的迭代成本，我们先在 V100 CUDA runtime 上进行策略搜索。测试使用 256M LoRA LLM 和原版视觉投影模块（mmproj），覆盖多组长 prompt 与图像样本。每个策略测试 60 条长输出样本。

原始结果路径：

```
/home/zcq/zcq_workspace/AICAS_eval/results/
  cuda_kv128_policy_sweep_mixed_20260601_1320/report_quality_rerank_20260613.json
```

需要说明的是，CUDA tok/s 仅用于同组参考，不作为最终板端性能指标。不同采样温度和运行条件下 tok/s 差异较大，最终策略选择主要依据 completion、Repeat-n 和 severe loop，板端性能以 KV260 实测为准。

结论是：KV 越小并不必然越好。b96 虽然能完成生成，但 severe loop 比例升高。

在不超过 128 token 的候选中，结构化保留策略说明了一个重要现象：token 分配会明显影响长输出重复率，不能只看 KV budget 数字。这些结果保留为质量搜索证据，不作为最终板端源码实现参数。最终稳定提交源码采用 tail84 路径，因为它实现简单、位置连续、server slot 状态容易维护，并且在 KV260 日志中可以直接验证物理 compact 生效。

6 4096 Token 长输出压力测试

我们还用约 501 prompt tokens 的仿官方长 prompt 做了 4096 token 单样本压力测试。这个测试能暴露 1024 token 内不明显、但 4096 token 时会出现的长循环退化。

表 1: CUDA 侧策略搜索汇总

策略	KV 预算	完成率	good	severe loop	平均 Repeat-4
b128/m64/p32/r28/t0.07	128	60/60	42/60	8/60	0.169
b128/m64/p32/r28/t0.05	128	60/60	29/60	11/60	0.217
b112/m48/p32/r28/t0.03	112	60/60	21/60	11/60	0.255
b96/m40/p28/r24/t0.03	96	60/60	28/60	16/60	0.239
b120/m56/p32/r28/t0.03	120	60/60	21/60	15/60	0.277
b112/m40/p40/r28/t0.03	112	60/60	16/60	25/60	0.310
b128/m48/p40/r36/t0.03	128	60/60	16/60	25/60	0.328
b128/m64/p32/r28/t0.03	128	60/60	19/60	29/60	0.326

结果路径:

```
/home/zcq/zcq_workspace/AICAS_eval/results/cuda_deep_readability_4096_20260601_1226/
all_summary.json
/home/zcq/zcq_workspace/AICAS_eval/results/
cuda_256lora_ctx128_temp0_4096_extra_20260601_1235/all_summary.json
```

表 2: 4096 token 长输出代表性结果

配置	输出 token	decode tok/s	Repeat-4	Repeat-8
256M LoRA, b112/m48/p32/r28, temp0.3 + repeat penalty	4096	383.24	0.0073	0.0051
256M LoRA, b192/m80/p64/r44, temp0.3 + repeat penalty	4096	381.03	0.0084	0.0081
256M LoRA, b112/m48/p32/r28, temp0	4096	397.95	0.0892	0.0827
256M LoRA, b128/m48/p40/r36, temp0	4096	395.21	0.0455	0.0258
500M base, tail84, temp0	4096	303.52	0.0218	0.0215
500M Q8-pruned, tail84, temp0	4096	391.69	0.0514	0.0514

这组结果说明，长输出质量不能只看速度。部分配置 decode 很快，但会反复输出相同句式或模板。最终源码包采用 tail84 作为稳定板端路径；结构化 CUDA 搜索结果用于解释小 KV budget 下的质量风险和后续优化方向。

7 KV260 板端最终提交结果

本节列出最终提交 JSON 的板端实测结果，对应最终提交版本的 bitstream、runtime、server 运行路径和测评流程。

最终提交 JSON 的综合分为 **87.7567**（原始值 **87.7567024854788**）。主要原始指标如下：

对应归一化分项为：

R_prefill = 0.9436

表 3: 最终提交 JSON 的主要指标

指标	数值
准确率	24/30 = 0.8000
prefill	53.3806 tok/s
decode	39.2592 tok/s
能效	5.6139 tokens/J
TTFT slope	1.4411 ms/char
TTFT intercept	2646.0375 ms
最终综合分	87.7567

```

R_decode = 0.8586
R_energy = 0.8272
R_a      = 0.8799
R_b      = 0.9510
R_TTFT   = 0.9155
score    = 87.7567

```

最终成绩来自完整系统，包括 bitstream、llama-server/runtime、采样策略、运行环境和测评流程。KV cache 压缩部分用于说明对长输出 decode 的贡献：相比不做有效 KV 压缩的长上下文路径，该部分主要改善 4096 token 长输出 decode。最终 JSON 中的能效、TTFT 和综合分数需要结合完整系统一起解读。

8 为什么选择该策略

我们比较了三类方案：

1. **server-side tail84 裁剪**：保留 4 个 sink token 和最近 80 个 token，实现简单、位置连续，配合物理 compact 后可以在 KV260 日志中直接验证 `used\_max\_p1` 下降。
2. **论文式 graph merge**：信息保留能力更强，但当前 runtime 中实现复杂，在线聚类和非连续 KV 管理会引入额外开销和稳定性风险。
3. **结构化 sink/media/prompt/recent 保留**：在 CUDA 搜索中有质量探索价值，但需要更复杂的多段位置管理；本次稳定提交源码没有采用该路线作为主路径。

因此最终选择第一类方案作为稳定提交实现。它继承了论文中 `Nsink = 4` 的关键稳定性设计，同时保持连续 tail 和 server token 历史一致。相比只修改 metadata，这种方案更容易证明后端 attention 真的访问短 KV：

- sink tokens 保持 RoPE 和 attention 稳定性；
- recent tail 维持局部语言连续性；
- server token/media bookkeeping 避免多模态请求保留过期位置；

- 物理 compact 确保硬件实际访问短 KV，而不是只在逻辑上声明短窗口。

9 结论

本部分最终采用的长上下文压缩方法是：

```
enable server-side KV tail crop
crop when slot.n_past > 128
retain first 4 sink tokens
retain recent 80 tokens
physically compact retained K/V rows to low-address cache cells
```

实验结论如下：

- 单纯缩短 KV 不一定提升可用质量，过激裁剪会导致 4096 token 输出重复。
- CUDA 搜索说明，不同 token 分配会显著影响小 KV budget 下的长输出稳定性。
- 最终稳定源码采用 tail84：保留 4 个 sink token 和最近 80 个 token。
- 裁剪后有效 KV 长度约 84，并在超过 128 后再次触发，形成短窗口锯齿。
- 板端必须使用物理 compact，才能把逻辑压缩转化为真实长输出 decode 改善。
- 日志中的 `used\_max\_p1=129->84` 是物理 compact 生效的关键证据。
- 最终能效、TTFT 和综合分数需要结合完整 runtime、功耗采样和测评流程解读。
- KV crop 的直接收益体现在长输出 decode 阶段。

本方案证明，长上下文优化不能只停留在逻辑 metadata 裁剪上。最终提交路径通过 sink + recent tail 保留必要上下文，并通过 physical compact 把保留的 K/V 行搬移到低地址 cache cell，确保后端 attention 使用范围真实下降，并首先体现为长输出 decode 收益。

后续如果继续优化，可以把当前被丢弃的中间区域改成轻量 centroid merge，或重新引入更精细的 media/prompt anchor 保留，进一步接近论文的“merge, not discard”思路，在不明显增加硬件开销的情况下保留更多上下文信息。

参考文献

- [1] N. I. Bhattacharya, S. Sathvik, A. Yadav, A. Dixit, and B. Kumar, “A Graph-Based Methodology for Dynamic KV-Cache Compression in Transformer Inference,” in *Proceedings of the IEEE International Symposium on Circuits and Systems (ISCAS)*, Shanghai, China, 2026, Paper 1276, Session C5L-09. Available: https://epapers2.org/iscas2026/ESR/paper_details.php?paper_id=1276.